01 Что мы собираем

От запроса до покупки

Разбираем, как устроен магазин.
Затем собираем его по шагам.

Понять → Собрать → Проверить



↳ Начнём с одной покупки и проследим, что делает каждый участник.

Одна покупка — несколько действий

Аня уже вошла в аккаунт. Теперь она выбирает и покупает кроссовки.

01
Каталог
Найти North Runner

02
Размер
Выбрать 42

03
Корзина
Добавить 2 пары

04
Заказ
Подтвердить покупку

05
История
Увидеть свой заказ

Каждая стрелка — новое действие

Посмотреть товар, добавить позицию и оформить заказ — разные запросы.

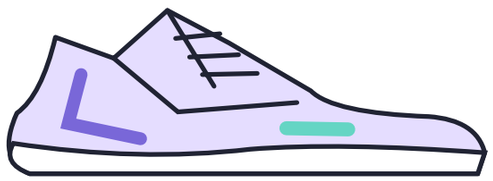
В корзине также есть City Walk × 1.
Итого: 2 × 7 990 + 5 990 = 21 970 ₽.

↳ Сервер нужен на каждом шаге, где мы читаем или меняем данные магазина.

Интерфейс показывает и отправляет

Кнопка связывает действие человека с запросом к серверу.

ЭКРАН ПОКУПАТЕЛЯ · ИЛЛЮСТРАЦИЯ



North Runner

7 990 ₽ за пару

42 ~~43~~

Выбрано: 2 пары

Добавить в корзину

1. **Показать данные**
   Название, цену и размеры интерфейс получил от API.
2. **Собрать выбор**
   Размер 42 соответствует записи с ID 101. Количество — 2.
3. **Отправить запрос**
   Сервер получит ID размера и количество, затем вернёт результат.

↳ Отключённая кнопка удобна пользователю. Правила покупки всё равно проверяет сервер.

01 Что мы собираем

Сервер проверяет правила магазина

Запрос — просьба выполнить действие. Сервер решает, допустимо ли оно.

Перед оформлением заказа

- ✓ Пользователь вошёл в аккаунт
- ✓ Корзина принадлежит ему
- ✓ Количество больше нуля
- ✓ На складе достаточно пар
- ✓ Сумма рассчитана по ценам из БД

Псевдокод · описывает идею — Учебный пример

```
пользователь = проверить_доступ()
корзина = найти_его_корзину()
проверить_количества(корзина)
проверить_остатки(корзина)
сумма = рассчитать_по_ценам_базы()
оформить_заказ(корзина, сумма)
```

↳ Пользователь сообщает свой выбор. Сервер проверяет его и рассчитывает результат.

База помнит то, что мы сохранили

Приложение можно перезапустить. Записанный заказ должен остаться.

ПРИЛОЖЕНИЕ

Обрабатывает запросы.
Временные значения живут в памяти.

перезапуск ↻ память процесса заново

После запуска

Приложение снова читает сохранённые данные из PostgreSQL.

СОХРАНЁННЫЙ ЗАКАЗ · ФРАГМЕНТ ТАБЛИЦЫ

id	501
user_id	7 · Аня
total	21970.00
status	created

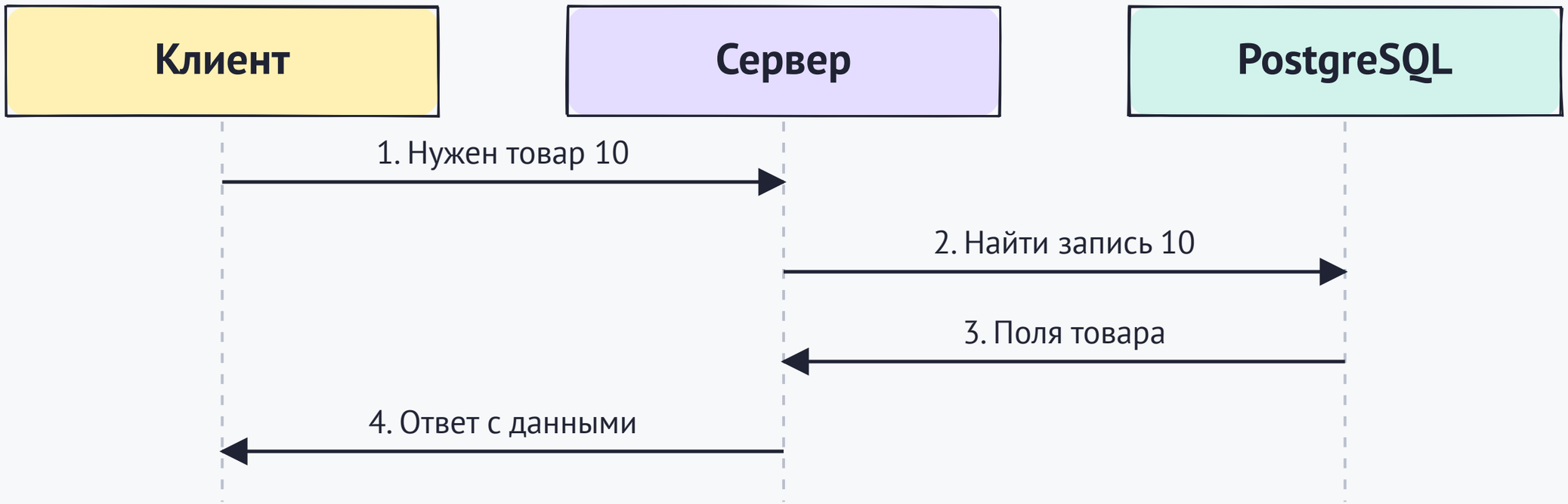
Каталог, пользователи, корзины и заказы хранятся в своих таблицах.

↳ Память приложения и постоянное хранилище решают разные задачи.

01 Что мы собираем

Клиент, сервер и база

Проследим чтение одной карточки товара.



↳ В этой архитектуре клиент обращается к API, а с базой работает сервер.

01 Что мы собираем

У каждой технологии своя работа

Пока достаточно понять место инструмента. Код разберём в следующих главах.

Запустить

Python + venv
Код и его окружение

Docker Compose
Запуск PostgreSQL

.env
Локальные настройки

Обработать запрос

FastAPI + Swagger
Маршруты и проверка API

Pydantic
Входные данные и ответы

passlib + python-jose
Пароли и токены доступа

Работать с данными

PostgreSQL
Хранение и запросы

SQLAlchemy
Работа с БД из Python

Alembic
Изменения схемы БД

↳ Инструменты складываются в одну систему: запуск → запрос → правила → данные.

Разбираем и сразу проверяем

Каждая тема приводит к небольшому действию, которое можно повторить.

01

Понять

Какую задачу решаем

02

Увидеть

Схема и данные

03

Прочитать

Код или псевдокод

04

Проверить

Что получилось

05

Применить

Шаг практики

Проверь себя перед второй главой

Где хранится заказ? Кто проверяет остатки?
Что отправляет клиент после нажатия кнопки?

↳ Следующая глава: читаем запрос и ответ так же уверенно, как сообщение человеку.

API — способ обратиться к программе

Мы заранее договариваемся, какой запрос отправить и какой ответ получить.

ПРОСЬБА КЛИЕНТА

«Покажи товары»

GET /products

Адрес и метод определяют операцию API.

JSON · ожидаемый фрагмент ответа — Учебный пример

```
[
  {
    "id": 10,
    "name": "North Runner",
    "price": "7990.00"
  }
]
```

↳ API задаёт понятный программам договор: вход, результат и возможные ошибки.

Разложим запрос на части

Аня добавляет две пары размера 42 в свою корзину.

HTTP · сокращённая запись запроса — Учебный пример

```
POST /cart/items HTTP/1.1
Host: localhost:8000
Content-Type: application/json
Authorization: Bearer <токен>

{
  "product_size_id": 101,
  "quantity": 2
}
```

1. **Метод: POST**
   Какую операцию просим выполнить.
2. **Путь: /cart/items**
   С какой частью магазина работаем.
3. **Заголовки**
   Адрес сервера, формат данных и доступ.
4. **Тело: JSON**
   Какой размер добавить и сколько пар.

↳ Метод + путь + заголовки + тело. Эти части читаем отдельно.

Как прочитать адрес API

URL показывает, куда обратиться и какие параметры передать.

```
http://localhost:8000/products?size=42
```

`http://`

Схема

Как обращаемся

`localhost`

Адрес узла

Этот компьютер

`:8000`

Порт

Вход в приложение

`/products`

Путь

Каталог

`?size=42`

Параметры

Фильтр размера

↳ На локальном стенде API и PostgreSQL слушают разные порты.

Путь выбирает конкретный товар

Число в адресе — идентификатор записи.

GET `/products/10`

ТАБЛИЦА PRODUCTS · ФРАГМЕНТ

id	name	price
10	North Runner	7990.00
20	City Walk	5990.00

/products/{id}

Шаблон маршрута. Вместо `{id}` клиент подставляет значение.

`/products/10` → North Runner
`/products/20` → City Walk

↳ ID товара, название товара и размер обуви — разные значения.

Параметры уточняют выборку

Меняйте фильтры: адрес и ожидаемый результат изменяются вместе.

Бренд **North · id 1** Размер **42** Учебная модель

GET `/products?brand_id=1&size=42`

Ожидаемый ответ: 200 **· 1 товар**

Товар	Цена	Размер · остаток
North Runner	7 990 ₽	42 · 3 пары

↳ После «?» идут параметры. Символ «&» соединяет несколько условий.

Тело передаёт данные изменения

Путь выбирает позицию корзины. JSON сообщает новое количество.

HTTP · сокращённая запись запроса — Учебный пример

```
PATCH /cart/items/55
Content-Type: application/json
Authorization: Bearer <токен>

{
  "quantity": 3
}
```

К КАКОЙ ЗАПИСИ

55 — ID позиции в корзине.
Он не равен ID товара или размера.

ЧТО ИЗМЕНИТЬ

quantity: 3 — установить три пары.
Было 2 → станет 3.

Сервер также проверит владельца позиции.

↳ Path: какую запись. Body: какие значения. Authorization: кто обращается.

02 Как читать запросы и ответы

JSON — данные с понятной структурой

Скобки, имена полей и типы помогают прочитать сообщение.

JSON · сокращённая карточка товара — Учебный пример

```
{
  "id": 10,
  "name": "North Runner",
  "sizes": [42, 43],
  "image_url": null
}
```

1 **{ } — объект**
Поля с именами: id, name, sizes, image_url.

2 **[ ] — массив**
Список значений. Здесь — два размера.

3 **"Текст", 10, null**
Строка, число и отсутствие значения.

4 **true / false — boolean**
Логическое значение. Например, is_admin в публичном профиле.

↳ Корректный JSON ещё не означает, что данные подходят нашему API.

Ответ тоже состоит из частей

Сначала узнаём исход запроса, затем читаем данные.

HTTP · сокращённый ожидаемый ответ — Учебный пример

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "name": "North Runner",
  "price": "7990.00"
}
```

1. **Статус: 200**
   Запрос выполнен успешно.
2. **Заголовки**
   Content-Type сообщает формат тела.
3. **Тело ответа**
   Поля карточки, которые получил клиент.

↳ Ответ нужно читать целиком: статус объясняет исход, тело — подробности.

02 Как читать запросы и ответы

Метод говорит, что мы хотим сделать

Один путь может поддерживать несколько разных операций.

Метод	Пример	Что просим
GET	`/products/10`	Прочитать карточку
POST	`/products`	Создать товар · администратор
PUT	`/products/10`	Заменить редактируемые поля карточки
PATCH	`/cart/items/55`	Изменить указанное поле: quantity
DELETE	`/cart/items/55`	Удалить позицию из своей корзины

↳ Для PUT передаём полный набор редактируемых полей. Для PATCH — выбранные изменения.

Адрес называет ресурс магазина

REST помогает организовать API вокруг данных и действий над ними.

`/products`	Каталог товаров
`/brands`	Список брендов
`/cart`	Своя корзина
`/orders`	Свои заказы

Коллекция

`GET /products`

Список карточек

Одна запись

`GET /products/10`

Карточка с ID 10

↳ Адрес выбирает ресурс, метод — действие, контракт — допустимые данные и результат.

Успех бывает разным

Статус помогает понять, что именно завершилось.

200 OK	201 Created	204 No Content
Получено	**Создано**	**Удалено**
`GET /products/10`	`POST /orders`	`DELETE /cart/items/55`
Карточка найдена. В теле — её данные.	Появился заказ. В теле — результат.	Позиция удалена. Тела ответа нет.
`{"id": 10, …}`	`{"id": 501, …}`	Пустое тело

↳ Это выбранные ответы нашего API. После 204 не пытаемся читать JSON.

Отказ тоже должен быть понятным

Одинаковая красная надпись скрывает разные причины.

401	Нет действительных данных входа	Корзина без токена
403	Недостаточно прав	Обычный пользователь создаёт товар
404	Ресурс не найден	Неизвестный ID товара
409	Конфликт с текущим состоянием	Заказываем 2 пары, осталась 1
422	Входные данные не прошли проверку	quantity = 0 при требовании больше 0

409 для нехватки остатка — выбранное соглашение магазина. Тело ответа объясняет причину.

↳ Сначала определяем причину отказа, затем решаем, что исправлять.

Иногда проблема на стороне сервера

Неожиданный сбой программы требует диагностики.

КЛИЕНТ ВИДИТ

500

Не удалось выполнить запрос.

Что выясняем

Какой запрос отправили?
Что ожидали?
Где возникло исключение?

Python traceback · условный пример — Учебный пример

```
Traceback (most recent call last):
  File "services/catalog.py", line 18
    return product.name
AttributeError:
  'NoneType' has no attribute 'name'
```

↳ Статус сообщает о сбое. Причину ищем в серверной диагностике и воспроизводимом запросе.

Теперь прочитай запрос сам

Назови действие, ресурс и каждое условие. Затем открой разбор.

```
GET /products?brand_id=1&size=42&max_price=8000
```

Читаем список товаров. Бренд — 1, размер — 42, цена — не выше 8 000 ₽. В наших данных подходит North Runner за 7 990 ₽.

Если совпадений нет: **200** и пустой список `[]`. Этот запрос не создаёт заказ и не списывает остаток.

Условия: бренд 1, размер 42, цена до 8 000 включительно. Ответ объясняет выбранный учебный контракт.

↳ Дальше научимся читать Python-код, который обрабатывает такие запросы.

Функция превращает вход в результат

Начнём с цены одной позиции: две пары по 7 990 ₽.

Python · запускаемый пример

```
def line_total(price, quantity):
    return price * quantity

result = line_total(7990, 2)
print(result)
```

price = **7990**

quantity = **2**

7990 × 2

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
15980
```

Результат вернулся в переменную `result`.

↳ def объявляет функцию. Вызов запускает её. return возвращает вычисленное значение.

Список хранит позиции, словарь — поля

Одна позиция корзины состоит из ID размера и количества.

Python · запускаемый пример

```
cart = [
    {"product_size_id": 101, "quantity": 2},
    {"product_size_id": 201, "quantity": 1},
]
for item in cart:
    print(item["product_size_id"],
item["quantity"])
```

cart — список из двух словарей

`for` берёт очередную позицию. `item["quantity"]` читает её поле.

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
101 2
201 1
```

Каждая строка вывода: ID размера, затем количество.

↳ В памяти это объекты Python. Для передачи по HTTP их преобразуют в JSON.

Аннотация описывает ожидаемый тип

Обычный Python сам не отклоняет аргумент только из-за аннотации.

Python · запускаемый пример

```python
def twice(quantity: int) -> int:
    return quantity * 2

print(twice(2))
print(twice("2"))
```

quantity: int → ожидаем число

→ `int` в записи функции также описывает ожидаемый результат.

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
4
22
```

4 — число.
22 — строка: `"2" * 2` повторяет текст.

↳ Аннотации помогают читать и проверять код. Проверку входа API выполняет отдельный механизм.

Класс описывает объект с данными

Объект хранит своё состояние в атрибутах.

Python · запускаемый пример

```
class Product:
    def __init__(self, name):
        self.name = name

shoe = Product("North Runner")
print(shoe.name)
```

ОБЪЕКТ SHOE В ПАМЯТИ

Product

`name` → North Runner

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
North Runner
```

↳ Обычный класс ещё не создаёт таблицу и не проверяет входные данные.

Импорт соединяет файлы программы

main-файл собирает приложение из небольших частей.

Файлы учебного приложения — Учебный пример

```
sneaker-examples/
├── demo.py
├── schemas.py
├── services.py
└── routers/
    ├── __init__.py
    └── catalog.py
```

Python · фрагмент demo.py

```python
from fastapi import FastAPI
from routers import catalog

app = FastAPI()
app.include_router(catalog.router)
```

`catalog` — импортированный модуль.
`catalog.router` — объект внутри него.

↳ Импорт даёт доступ к определениям. include_router подключает операции к приложению.

Исключение прерывает обычный путь

Недопустимое количество превращаем в явную ошибку.

Python · запускаемый пример

```
def check_quantity(quantity):
    if quantity <= 0:
        raise ValueError("Нужно больше нуля")
    return quantity

try:
    check_quantity(0)
except ValueError as error:
    print(error)
```

raise → поиск обработчика

При quantity = 0 строка `return quantity` уже не выполнится.

✓ ПРОВЕРЕННЫЙ ВЫВОД

Нужно больше нуля

`except ValueError` поймал ошибку и вывел её текст.

↳ Исключение Python и HTTP-ответ — разные вещи. Их связывает обработка ошибки.

Деньги считаем с Decimal

Десятичную цену создаём из строки, сохраняя исходные цифры.

Python · запускаемый пример

```python
from decimal import Decimal

print(0.1 + 0.2)
print(Decimal("0.1") + Decimal("0.2"))
total = Decimal("7990.00") * 2
print(total)
```

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
0.30000000000000004
0.3
15980.00
```

Decimal("7990.00")

Сначала точное значение цены.
Потом умножение на количество.

↳ float хранит двоичное приближение. Для цен используем согласованную десятичную арифметику.

Пройди по коду самостоятельно

Для каждого вызова проверь условие и найди строку return.

Python · запускаемый пример

```
def can_buy(stock, quantity):
    if quantity <= 0:
        return False
    return quantity <= stock

print(can_buy(3, 2))
print(can_buy(3, 4))
print(can_buy(3, 0))
```

Проверенный вывод

```
True
False
False
```

2 ⩽ 3 → True.
4 ⩽ 3 → False.
0 ⩽ 0 → ранний return False.

↳ Сначала подставь значения. Затем следуй только по той ветке, условие которой выполнено.

Кто запускает FastAPI

Uvicorn принимает сетевое соединение и передаёт запрос приложению.



ИЗ ПАПКИ PRESENTATION/SNEAKER-EXAMPLES · ПОСЛЕ УСТАНОВКИ ЗАВИСИМОСТЕЙ

```
python -m uvicorn minimal:app --port 8000
```

↳ minimal — файл minimal.py. app — объект FastAPI внутри этого файла.

Минимальное приложение — семь строк

Этот файл можно запустить и проверить настоящим HTTP-запросом.

Python · запускаемый пример

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}
```

1. **Импортировать FastAPI**
   Получить класс из установленной библиотеки.
2. **Создать app**
   Объект приложения хранит маршруты.
3. **Зарегистрировать /health**
   GET по этому пути вызывает health().
4. **Вернуть данные**
   Словарь станет JSON-ответом клиенту.

↳ Проверенный результат GET /health: 200 и {"status": "ok"}.

Метод и путь выбирают обработчик

Декоратор связывает HTTP-операцию с Python-функцией.

GET `/health`

Python · фрагмент minimal.py

```python
@app.get("/health")
def health():
    return {"status": "ok"}
```

Совпали метод и путь

Вызываем `health()`.
Получаем 200.

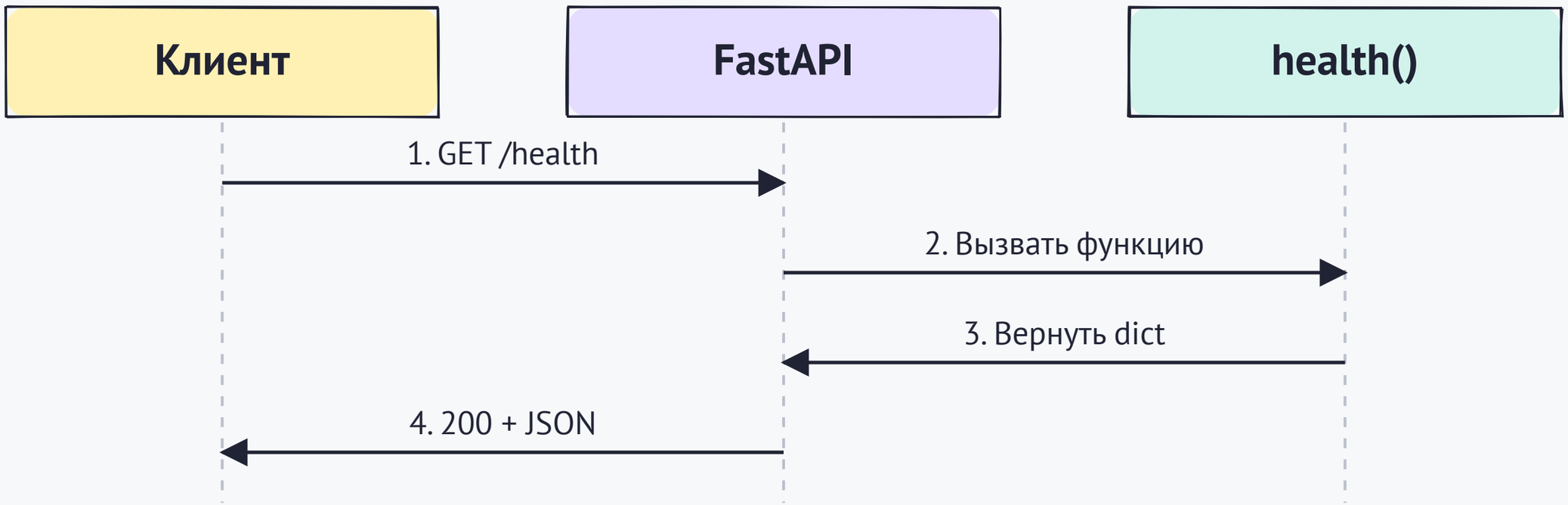
`POST /health` → 405
Путь есть, метод не поддерживается.

`GET /missing` → 404
Такого маршрута в примере нет.

↳ Имя функции — для Python. Внешний адрес задаётся в декораторе.

Проследим один запрос до ответа

Внутри обработчика работаем с Python-данными. Клиент получает HTTP.



↳ Запрос → выбор функции → return → сериализация → ответ.

Проверяем запрос через Swagger

Открой /docs у запущенного учебного приложения.

СХЕМА ИНТЕРФЕЙСА /DOCS

GET /health

1. Открыть операцию
2. Нажать **Try it out**
3. Нажать **Execute**

200 `{"status": "ok"}`

1. **Проверить адрес**
   http://127.0.0.1:8000/docs
2. **Прочитать Server response**
   Статус и тело после Execute — фактический ответ.
3. **Отличить пример от ответа**
   Example Value показывает образец схемы. Он не доказывает выполнение запроса.

↳ Swagger отправляет обычный HTTP-запрос. Это наш клиент для практики.

Pydantic проверяет данные на входе

Схема задаёт поля, типы и ограничения запроса.

Python · запускаемый пример

```python
from pydantic import BaseModel, Field

class CartItemCreate(BaseModel):
    product_size_id: int = Field(gt=0)
    quantity: int = Field(gt=0, strict=True)
```

Field(gt=0, strict=True)

Целое число больше нуля.
Строку «2» и boolean не принимаем.

✓ ПРОВЕРЕННЫЙ ВВОД

```
{"product_size_id": 101, "quantity": 2}
```

Такое тело запроса проходит схему.

↳ Форма данных проверена. Наличие размера и остаток ещё предстоит проверить в базе.

Неверный ввод не попадает в обработчик

Переключай количество и сравни проверенные ответы FastAPI.

quantity 0 · ноль

JSON · вход /demo/cart/items

```
{
  "product_size_id": 101,
  "quantity": 0
}
```

422

```
{
  "loc": ["body","quantity"],
  "type": "greater_than",
  "ctx": {"gt":0}
}
```

Тело обработчика: **не вызвано**

quantity должно быть больше 0.

↳ 422 объясняет, где вход не прошёл схему. Вызов тела обработчика проверяем отдельно.

Схема ответа выбирает видимые поля

Внутренний объект может содержать больше данных, чем получает клиент.

Python · схема и сокращённый обработчик

```python
class UserRead(BaseModel):
    id: int
    email: str

@app.get("/demo/user",
         response_model=UserRead)
def example_user():
    return internal_user
```

`id` `email` ~~`hashed_password`~~

✓ ПРОВЕРЕННЫЙ JSON-ОТВЕТ

```
{
  "id": 7,
  "email": "anya@example.com"
}
```

↳ response_model=UserRead фильтрует ответ. Пароль и его хэш клиенту не возвращаем.

На входе и выходе разные данные

Клиент задаёт свойства товара. Сервер добавляет созданный ID.

ЗАПРОС · PRODUCTCREATE

`name` `price`

```
{"name": "North Runner",
 "price": "7990.00"}
```

Название непустое.
Цена — Decimal больше 0.

→

ОТВЕТ · PRODUCTREAD

`id` `name` `price`

```
{"id": 10,
 "name": "North Runner",
 "price": "7990.00"}
```

201 Сервер сформировал ответ.

↳ ProductCreate описывает вход. ProductRead описывает публичный результат.

Depends передаёт готовое значение

FastAPI вызывает зависимость и подставляет её результат в обработчик.

Python · фрагмент demo.py

```python
from fastapi import Depends, Query
def page_size(
    limit: int = Query(10, ge=1, le=50)
):
    return limit

@app.get("/demo/page")
def page(limit: int = Depends(page_size)):
    return {"limit": limit}
```

1 Запрос `?limit=3`

2 `page_size` проверит и вернёт 3

3 `page` получит `limit=3`

↳ Передаём Depends функцию page_size, а не результат её ручного вызова.

Роутер собирает связанные операции

Каталог получает свой файл и общий префикс адресов.

Python · сокращённый routers/catalog.py

```python
from fastapi import APIRouter
from services import list_products

router = APIRouter(prefix="/products")

@router.get("")
def catalog():
    return list_products()
```

`/products` + `""` = `/products`

В demo.py

```
app.include_router(catalog.router)
```

Позже отдельные роутеры:
auth · cart · orders

↳ APIRouter группирует маршруты. include_router делает их частью приложения.

Роутер принимает запрос, сервис решает задачу

Для оформления заказа нам понадобятся разные уровни ответственности.

Роутер

Принять HTTP-запрос.
Вызвать сервис и вернуть ответ.

Псевдокод · будущий POST /orders — Учебный пример

```
пользователь = проверить_доступ()
заказ = сервис.оформить(пользователь)
вернуть_ответ(заказ)
```

Сервис заказа

- ✓ Прочитать свою корзину
- ✓ Проверить остатки
- ✓ Рассчитать сумму
- ✓ Сохранить заказ и списать склад
- ✓ Очистить корзину

Изменения объединит транзакция.

↳ Дальше подключим PostgreSQL и разберём, как эти действия меняют реальные данные.

Из чего состоит локальный запуск

Python работает на компьютере. PostgreSQL — в контейнере Docker.



↳ Два процесса общаются по сети. У каждого свои зависимости, настройки и журнал ошибок.

venv изолирует библиотеки проекта

Одна папка проекта — своё окружение и свой набор зависимостей.

Терминал · macOS / Linux — Учебный пример

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
python -c "import sys;
print(sys.executable)"
```

До активации

`python` выбирается из системного PATH.

После активации

`python` указывает на `.venv/bin/python`.

↳ python -m pip устанавливает пакеты для того Python, которым вы запускаете приложение.

requirements фиксирует зависимости

Сохраняем проверенные версии, чтобы повторить окружение.

Фрагмент проверенного requirements.txt — Учебный пример

```
fastapi==0.141.1
pydantic==2.13.5
uvicorn==0.52.4
httpx==0.28.1
```

Установка из файла — Учебный пример

```
python -m pip install -r
requirements.txt
python -m pip check
```

Для полного воспроизведения

Фиксируем также транзитивные пакеты и версию Python.

↳ Файл зависимостей — рецепт установки. Его совместимость подтверждается запуском и проверками.

Образ — шаблон для контейнера

postgres:17 содержит подготовленное окружение PostgreSQL.

ОБРАЗ POSTGRES:17

- PostgreSQL
- Системные библиотеки
- Инструкция запуска

Скачать образ

Учебный пример

```
docker pull postgres:17
```

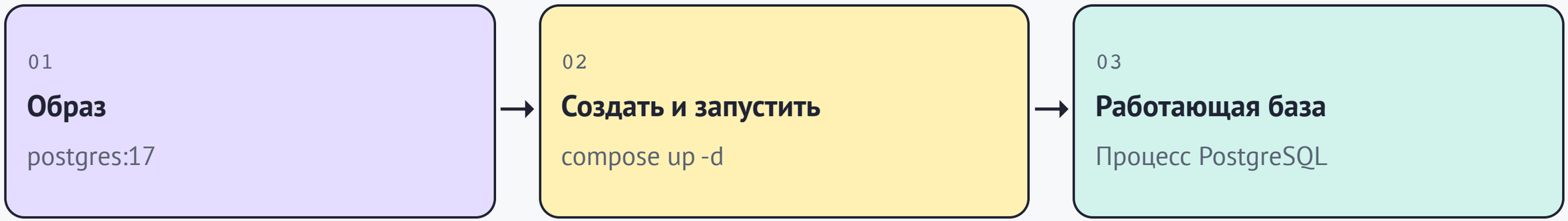
Образ не равен данным магазина

Из одного образа можно запустить несколько отдельных контейнеров.

↳ Образ содержит файлы и настройки запуска. Работающий процесс появится в контейнере.

Контейнер — запущенный экземпляр

У контейнера есть состояние, сеть и журнал процесса.



Команды из папки с compose.yaml — Учебный пример

```
docker compose ps
docker compose logs --tail 30 db
docker compose stop db
```

↳ stop останавливает процесс. Следующий up -d снова поднимает сервис.

Compose описывает запуск в файле

Один сервис db: образ, переменные, порт и место хранения.

compose.yaml · фрагмент — Учебный пример

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:55432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
```

Отступы задают структуру

db — имя сервиса.
pgdata — именованный том.

Запуск полного файла из примеров — Учебный пример

```
docker compose config --quiet
docker compose up -d --wait
```

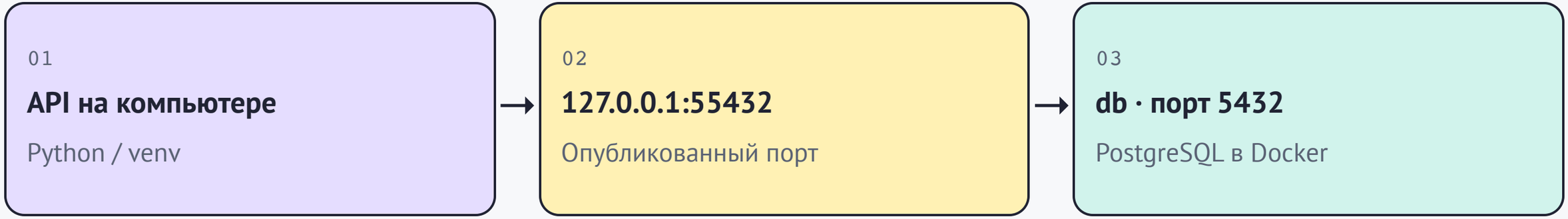
↳ Полный файл рядом с презентацией также объявляет pgdata, пользователя, базу и healthcheck.

05 Окружение и Docker

Адрес зависит от места запуска API

localhost всегда означает текущее сетевое окружение.

API запущен: На компьютере — наш стенд



Публикация порта: `127.0.0.1:55432:5432` · компьютер → контейнер

↳ С компьютера: 127.0.0.1:55432. Между сервисами одной сети Compose: db:5432.

Том переживает замену контейнера

Файлы PostgreSQL находятся в именованном томе pgdata.

Контейнер A

PostgreSQL пишет файлы базы

Контейнер B

После пересоздания читает те же файлы

↔

Том pgdata

Каталог данных PostgreSQL
таблицы · записи · индексы

В PostgreSQL 17 монтируем том в `/var/lib/postgresql/data`.

↳ Пересоздание контейнера и удаление тома — разные операции. Том не заменяет резервную копию.

DATABASE_URL — адрес подключения

Одна строка отвечает на вопросы: чем, кем, куда и в какую базу.

Учебный адрес · пароль вымышленный — Учебный пример

```
postgresql://lesson:lesson-only@127.0.0.1:55432/sneaker_lesson
```

`postgresql`	`lesson`	`lesson-only`	`127.0.0.1:55432`	`sneaker_lesson`
Протокол	Пользователь	Пароль	Хост и порт	Имя базы

↳ Один хост может обслуживать несколько баз. Имя базы — не имя таблицы и не имя контейнера.

.env хранит настройки окружения

Пример файла объясняет формат. Реальный файл подставляет значения.

.env.example · учебные значения — Учебный пример

```
POSTGRES_USER=lesson
POSTGRES_PASSWORD=lesson-only
POSTGRES_DB=sneaker_lesson
DB_PORT=55432
```

Сначала скопировать — Учебный пример

```
cp .env.example .env
```

Compose

Читает .env для подстановки `${...}` в YAML.

Python

Нужна явная загрузка настроек: например, `load_dotenv()` перед `os.environ["DATABASE_URL"]`.

↳ .env исключаем из Git. .env.example коммитим без реальных секретов.

API запущен — а база доступна?

Проверка готовности должна действительно обратиться к PostgreSQL.

Псевдокод GET /health — Учебный пример

```
попробовать:
    открыть соединение с таймаутом
    выполнить SQL: SELECT 1
    закрыть соединение
    вернуть 200, {status: "ok"}
если база недоступна:
    вернуть 503, {status: "unavailable"}
```

GET /live → 200

Процесс API отвечает. Базу не проверяем.

GET /health → 200

Наш readiness-маршрут проверил соединение и простой SQL-запрос.

↳ SELECT 1 проверяет соединение и выполнение SQL. Наличие таблиц и миграций проверяем отдельно.

Если не запускается: идём по цепочке

Проверяем ближайшую причину, затем повторяем тот же запрос.

1. Docker недоступен	`docker info`	Запустить Docker Desktop / engine
2. Контейнер не healthy	`docker compose ps`	Прочитать logs db; проверить порт
3. Нет соединения	Хост → порт → база	Сверить адрес с местом запуска API
4. Отказ в доступе	Роль и пароль	Проверить учётные данные уже созданной БД
5. Модуль не найден	`python -m pip --version`	Проверить venv и requirements

↳ Ошибка — подсказка, на каком участке оборвалась цепочка. Исправляем одну причину за раз.

Почему списка Python недостаточно

Каталог должен пережить перезапуск и быть общим для всех покупателей.

Память одного процесса — Учебный пример

```
products = [
    {"id": 10, "stock": 3}
]
# После перезапуска создаётся заново
```

Несколько процессов

У каждого — своя память и своя копия списка.

PostgreSQL

Хранит данные на диске и обслуживает разные соединения.

Правила согласованности

Ограничения и транзакции помогают сохранять корректные данные.

↳ База хранит общее состояние. Без корректных запросов и транзакций она не решает гонки автоматически.

Таблица: строки и столбцы

Строка описывает один товар. Столбец хранит один вид значения.

PRODUCTS

id	name	brand_id	price
10	North Runner	1	7990
20	City Walk	2	5990
30	North Trail	1	9990

3 строки → 3 товара

price → цена каждой строки

✓ Выполнено в PostgreSQL

↳ Таблица задаёт общую структуру. Каждая запись получает свои значения полей.

Тип поля объясняет его значение

Цена, количество, текст и время требуют разных правил хранения.

`text`

name

North Runner

Текст

`numeric(10,2)`

price

7990.00

Точное десятичное число

`integer`

stock

3

Целое количество

`boolean`

is_admin

false

Да / нет

`timestamptz`

created_at

2026-10-01 12:00+00

Момент времени

`NULL`

image_url

нет значения

Не пустая строка и не 0

↳ Тип ограничивает форму значения. CHECK и NOT NULL добавляют правила к этому типу.

Первичный ключ отличает записи

Название может повториться. ID должен однозначно найти нужную строку.

ОТДЕЛЬНЫЙ МИНИ-ПРИМЕР · ОДИНАКОВЫЕ НАЗВАНИЯ

id	name
10	Runner
20	Runner

PRIMARY KEY · первичный ключ — Учебный пример

```
id integer GENERATED BY DEFAULT
AS IDENTITY PRIMARY KEY
```

Найти одну конкретную запись

```
WHERE id = 20
```

Название для этого ненадёжно.

↳ PRIMARY KEY означает уникальность и запрет NULL. ID не обязан идти без пропусков.

Семь таблиц — семь задач магазина

Разделяем сведения, которые меняются и используются по-разному.

`users`
Кто покупает

`brands`
Кто выпускает

`products`
Что продаём

`product_sizes`
Размер и остаток

`cart_items`
Что выбрано сейчас

`orders`
Факт покупки

`order_items`
Состав и цена покупки

↳ Связи соединяют таблицы обратно: от позиции корзины можно дойти до размера, товара и бренда.

Пользователи и бренды: первые записи

ID нужен для связей. Остальные поля описывают сущность.

USERS

id	email	is_admin
7	anya@example.com	false
8	boris@example.com	false

BRANDS

id	name
1	North
2	City

↳ UNIQUE(email) запрещает повтор значения email. UNIQUE(name) для бренда — решение нашего примера.

Товар и размер — разные сущности

Цена относится к карточке. Остаток относится к её размеру.

PRODUCTS

id	name	price
10	North Runner	7990

PRODUCT_SIZES

id	product_id	size	stock
101	**10**	42	3
102	**10**	43	0

↳ North Runner существует один раз. Его размеры 42 и 43 имеют независимые остатки: 3 и 0.

Внешний ключ связывает две таблицы

products.brand_id ссылается на существующий brands.id.

BRANDS

id	name
1	North
2	City

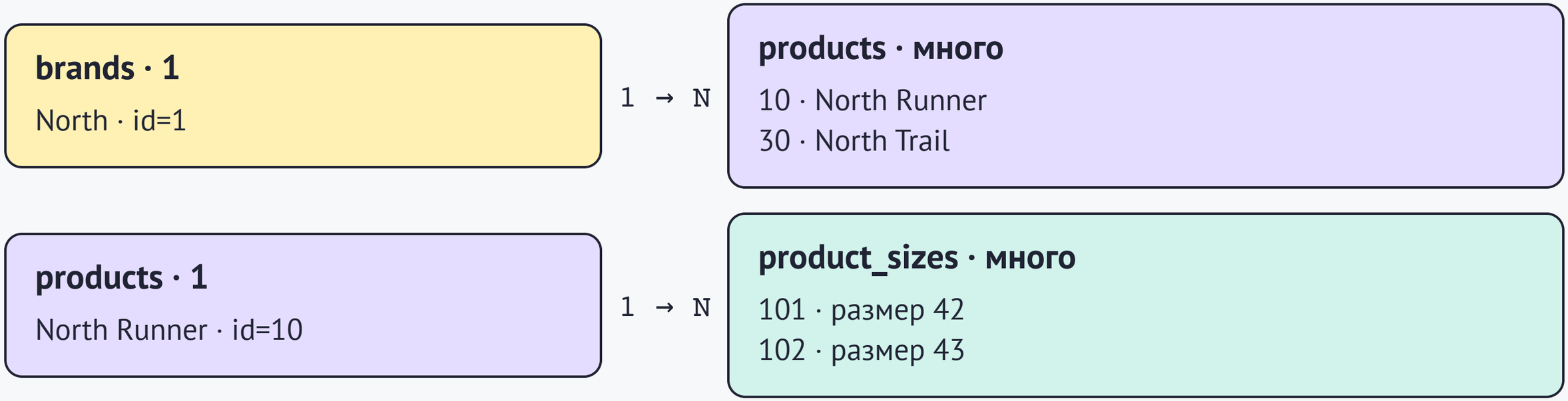
PRODUCTS

id	name	brand_id
10	North Runner	**1**
20	City Walk	**2**
30	North Trail	**1**

↳ brand_id = 1 → brands.id = 1 → North. Название бренда берём из связанной строки.

Один ко многим: где хранится ссылка

Внешний ключ находится на стороне «много».



↳ У многих товаров один brand_id. У многих размеров один product_id.

10, 42 и 101 — три разных значения

Чтобы положить размер в корзину, нужен ID записи размера.

01
product_id = 10
Товар North Runner

→

02
size = 42
Номер размера обуви

→

03
id = 101
Запись product_sizes

Тело запроса в корзину — Учебный пример

```
{"product_size_id": 101, "quantity": 2}
```

↳ В API передаём product_size_id=101. Число 42 само по себе не определяет товар.

Корзина хранит выбор пользователя

Одна строка: кто выбрал, какой размер и сколько пар.

CART_ITEMS

id	user_id	product_size_id	quantity
55	**7**	**101**	2
56	**7**	**201**	1
57	**8**	**102**	1

Пользователь 7 → позиции 55 и 56

Пользователь 8 → позиция 57

↳ Связь user_id хранит владельца. Проверку доступа к его строкам должен выполнять сервер.

Заказ сохраняет факт покупки

Шапка описывает покупку целиком. Позиции фиксируют её состав и цены.

orders · id=501

user_id: **7** · completed
total: **21 970 ₽**

Сумма при оформлении

2 × 7 990 + 1 × 5 990
= **21 970 ₽**

ORDER_ITEMS · ORDER_ID=501

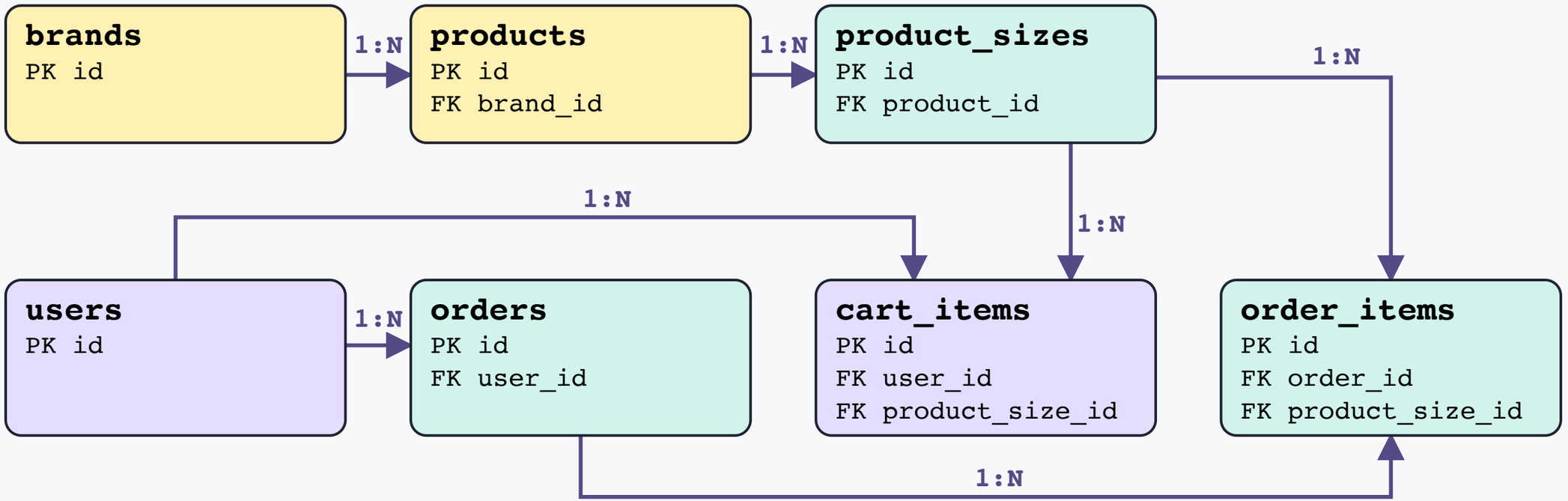
id	product_size_id	quantity	price_at_purchase
601	**101**	2	7990
602	**201**	1	5990

↳ price_at_purchase — цена одной пары на момент покупки. История не пересчитывается по текущему каталогу.

Вся схема магазина на одном экране

PK — первичный ключ. FK — внешний ключ. Стрелка: родитель → дочерние строки.

Подсветить связи: Вся схема 1:N — один ко многим



↳ Каждая стрелка — связь по ID. У родителя может быть 0…N дочерних строк.

Ограничения защищают саму базу

Проверки работают и при записи через API, и при прямом SQL.

NOT NULL `name = NULL`

Обязательное значение отсутствует

✓ Отклонено · 23502

UNIQUE Второй бренд `North`

Значение уже существует

✓ Отклонено · 23505

FOREIGN KEY `brand_id = 999`

Нет такого бренда

✓ Отклонено · 23503

CHECK `stock = −1`

Нарушено правило stock ⩾ 0

✓ Отклонено · 23514

↳ Pydantic проверяет вход API. Ограничения PostgreSQL защищают данные независимо от способа записи.

SELECT читает нужные строки

Выбираем поля, фильтруем, сортируем и ограничиваем результат.

Выборка: Цена до 8 000

✓ Выполнено в PostgreSQL

SQL — Учебный пример

```
SELECT id, name, price
FROM products
WHERE price <= 8000
ORDER BY price, id
LIMIT 10;
```

РЕЗУЛЬТАТ

id	name	price
20	City Walk	5990
10	North Runner	7990

↳ WHERE отбирает строки. ORDER BY задаёт порядок. LIMIT ограничивает количество строк.

JOIN соединяет строки по ключам

Добавим название бренда к каждому товару.

SQL · INNER JOIN — Учебный пример

```
SELECT p.id, p.name, b.name AS brand
FROM products AS p
JOIN brands AS b ON b.id = p.brand_id
ORDER BY p.id;
```

РЕЗУЛЬТАТ JOIN

id	name	brand
10	North Runner	**North**
20	City Walk	**City**
30	North Trail	**North**

↳ ON b.id = p.brand_id описывает совпадение ключей. p и b — короткие имена таблиц.

INSERT, UPDATE, DELETE меняют строки

Отдельный учебный бренд 99: добавили → переименовали → удалили.

1 · INSERT — Учебный пример

```
INSERT INTO brands(id, name)
VALUES (99, 'Demo');
```

ПОСЛЕ ШАГА

id	name
99	Demo

2 · UPDATE — Учебный пример

```
UPDATE brands
SET name = 'Demo Updated'
WHERE id = 99;
```

ПОСЛЕ ШАГА

id	name
99	Demo Updated

3 · DELETE — Учебный пример

```
DELETE FROM brands
WHERE id = 99;
```

ПОСЛЕ ШАГА

id	name

0 строк • [ ]

Показываем строку `WHERE id = 99` после выбранного шага.

↳ WHERE определяет, какие строки менять. Без него UPDATE и DELETE затронут все строки таблицы.

Практика: пройдите по связям

Позиция заказа 601. Найдите покупателя, товар, размер и бренд.

Кто: 601 → order 501 → user 7 → anya@example.com

Что: 601 → size 101 → product 10 → North Runner, размер 42

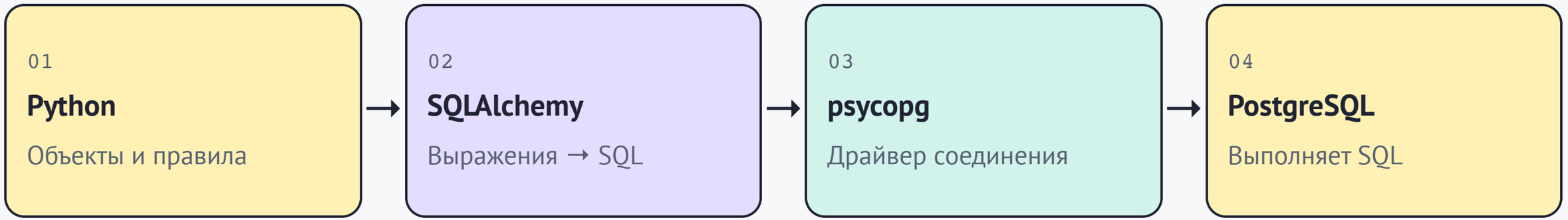
Бренд: product 10 → brand 1 → North

Цена покупки останется **7 990 ₽**. Текущая цена каталога её не заменяет.

↳ Готовы к следующему блоку: представить эти же таблицы и связи в Python через SQLAlchemy.

Как Python разговаривает с базой

SQLAlchemy связывает объекты Python и SQL-запросы.



`postgresql+psycopg://…`	База PostgreSQL + явно выбранный драйвер psycopg

↳ ORM упрощает работу с таблицами через объекты. SQL и ограничения базы продолжают действовать.

ORM-модель описывает таблицу

Класс Product связывает атрибуты Python со столбцами products.

models.py · сокращённый фрагмент — Учебный пример

```
class Product(Base):
    __tablename__ = "products"
    id: Mapped[int] = mapped_column(
        Identity(), primary_key=True)
    name: Mapped[str] =
mapped_column(Text)
    price: Mapped[Decimal] =
mapped_column(
        Numeric(10, 2))
```

Python	PostgreSQL
Product	products
id: int	integer · PRIMARY KEY
name: str	text · NOT NULL
price: Decimal	numeric(10,2)

↳ Класс задаёт соответствие. Таблицу в нашем приложении создаёт миграция.

ForeignKey и relationship: две роли

Ссылка защищает данные. Связь объектов помогает читать Python-код.

Product · фрагмент модели — Учебный пример

```
brand_id: Mapped[int] = mapped_column(
    ForeignKey("brands.id"))

brand: Mapped[Brand] = relationship(
    back_populates="products")
```

ForeignKey → правило БД

`brand_id=1` должен указывать на существующий `brands.id`.

relationship → объект Python

`product.brand.name`
→ **North**

↳ relationship не добавляет отдельный столбец brand. Связь в базе хранится через brand_id.

Engine и Session работают вместе

Engine управляет подключениями. Session сопровождает одну работу с данными.

database.py · идея настройки — Учебный пример

```
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(engine)

with SessionLocal() as session:
    product = session.get(Product, 10)
```

Engine · на приложение

Знает драйвер и адрес. Обычно управляет пулом соединений.

Session · на операцию / запрос

Отслеживает ORM-объекты и текущую транзакцию.

↳ Engine можно переиспользовать. Одну Session нельзя одновременно разделять между запросами.

SELECT возвращает объект или None

Ищем по первичному ключу: совпадений будет не больше одного.

Ищем товар: ID 10 · существует ✓ Проверенные результаты

SQLAlchemy 2.0 · фрагмент — Учебный пример

```
stmt = select(Product).where(
    Product.id == 10
)
product = session.scalar(stmt)
```

✓ РЕЗУЛЬТАТ POSTGRESQL

```
Product
id = 10
name = North Runner
price = Decimal("7990.00")
```

↳ Конструктор select описывает запрос. session.scalar выполняет его и берёт первое скалярное значение.

Таблица, схема API и JSON

Три представления одного товара решают разные задачи.

ORM · Product

Столбцы, ключи, связи.

```
price: Decimal
```

Pydantic · ProductRead

Поля публичного ответа.

```
from_attributes=True
```

JSON · ответ

Текст для клиента.

```
"price": "7990.00"
```

Проверенное преобразование — Учебный пример

```
dto = ProductRead.model_validate(product)
payload = dto.model_dump(mode="json")
```

↳ ORM-модель хранит данные, схема API выбирает контракт. Их поля не обязаны совпадать.

add, flush и commit — разные шаги

Посмотрим, когда появился ID и когда строку увидит другое соединение.

Шаг	ID	Виден другому соединению
new	None	Нет
add	None	Нет
flush	32	Нет
commit	32	Да

↳ flush отправляет изменения в текущую транзакцию. commit фиксирует её результат.

У сессии должно быть завершение

Успех фиксируем. Ошибку откатываем. Соединение освобождаем.

Отдельная операция · новая Session — Учебный пример

```
with Session(engine) as session:
    with session.begin():
        session.add(Brand(name="Demo"))
        session.flush()
        # Исключение → rollback
    # Успех → commit
# Выход → close
```

Ошибка после flush

Строка из опыта отсутствует после отката.

После неудачного flush

Перед повторным использованием той же Session вызываем `rollback()`.

↳ Граница транзакции должна иметь одного владельца. close не заменяет commit.

Изменить класс — мало

Код уже ожидает image_url, а в таблице такого столбца ещё нет.

Новая версия модели — Учебный пример

```
image_url: Mapped[str | None] = (
    mapped_column(Text)
)
```

Python ожидает

id · name · price · **image_url**

База пока на версии 0001

id · name · price

image_url отсутствует

✓ ПРОВЕРЕННЫЙ ОТКАЗ

```
SQLSTATE 42703
undefined_column
```

↳ Код и структура базы версионируются согласованно. Новый атрибут не выполняет ALTER TABLE.

Миграция хранит шаг изменения

Alembic применяет явные операции к структуре базы.

0002_image_url.py · исполняемый фрагмент — Учебный пример

```
def upgrade():
    op.add_column("products",
        sa.Column("image_url",
                  sa.Text(),
nullable=True))

def downgrade():
    op.drop_column("products",
"image_url")
```

Было · 0001

У products нет image_url.

Стало · 0002

Столбец появился.
У существующих строк — NULL.

↳ upgrade двигает схему вперёд. downgrade может удалить данные, поэтому его последствия читаем отдельно.

Версии складываются в цепочку

Файлы миграций описывают путь. База помнит применённую ревизию.



В папке учебных примеров — Учебный пример

```
python -m alembic history
python -m alembic current
python -m alembic upgrade head
```

alembic_version

После upgrade: **0002**
head — последняя ревизия нашей линейной цепочки.

↳ Миграция в репозитории и применённая миграция в базе — два разных состояния.

autogenerate предлагает черновик

Alembic сравнивает metadata с базой, а человек проверяет смысл изменений.

После изменения models.py — Учебный пример

```
python -m alembic revision --autogenerate
\
  -m "add image_url"
```

Перед upgrade читаем файл

Имена таблиц · типы · nullable
upgrade · downgrade · сохранность данных

Опасная догадка при rename

```
drop_column("name")
+ add_column("title")
```

Так можно потерять старые названия.

↳ Автогенерация видит различие структур. Намерение переименовать поле она надёжно не угадывает.

Seed добавляет строки после миграции

Сначала появляются таблицы, затем учебные бренды, товары и пользователи.

seed.py · идея вставки бренда — Учебный пример

```
stmt = insert(Brand).values(
    id=1, name="North"
).on_conflict_do_nothing()

with engine.begin() as connection:
    connection.execute(stmt)
```

Повтор seed	Бренды	Товары
Первый запуск	2	3
Второй запуск	2	3

Повторяемость имеет правила

Существующие строки сохраняются.
Seed не сбрасывает корзины и цены.

↳ Миграция управляет структурой. Seed даёт конкретные данные для разработки и проверки.

Слои помогают проследить запрос

Каждый файл отвечает за понятную часть пути от HTTP до данных.

`routers / api.py`	Принять запрос, зависимости, HTTP-ответ
`schemas.py`	Поля входа и публичного ответа
`services.py`	Правила сценария: что разрешено сделать
`models.py`	Таблицы и связи ORM
`database.py + core`	Сессии, подключения и настройки

GET /products/10 → роутер → поиск товара → Product → ProductRead → JSON

↳ Разделение файлов помогает читать и проверять код. Граница транзакции всё равно должна быть явной.

Зачем магазину знать пользователя

Аня и Борис обращаются к одному API, но работают со своими данными.

Аня · user_id=7

Корзина: позиции **55, 56**
Заказ: **501**

Борис · user_id=8

Корзина: позиция **57**
Заказ 501 ему не принадлежит.

Тело запроса не подтверждает личность

```
{"user_id": 7}
```

может написать любой клиент.

↳ Текущего пользователя сервер устанавливает по проверенным учётным данным, а не по заявленному user_id.

Кто ты — и что тебе разрешено

Аутентификация и авторизация отвечают на разные вопросы.

1 · Аутентификация

Токен проверен → пользователь **7**.
«Кто обращается?»

→

2 · Авторизация

Пользователь 7 владеет заказом 501.
«Разрешено ли это действие?»

Ситуация	Ответ в нашем API
Нет действительных данных входа	401 · требуется аутентификация
Вошёл, но нет роли администратора	403 · действие запрещено
Заказ чужой или отсутствует	404 · не раскрываем чужую запись

↳ Успешный вход подтверждает пользователя. Разрешение проверяется для каждой защищённой операции.

Регистрация создаёт пользователя

Пароль нужен для входа, но не должен попасть в публичный ответ.

01
Ввод
email + password

→

02
Проверки
Формат + уникальность

→

03
Хэш
bcrypt с солью

→

04
users
is_admin = false

POST /auth/register · пример — Учебный пример

```
{"email": "new@example.com",
 "password": "New-demo-N1!"}
```

✓ ФАКТИЧЕСКИЙ ОТВЕТ · 201

```
{
  "id": 11,
  "email": "new@example.com"
}
```

↳ Публичная регистрация не принимает is_admin. Повтор email запрещён также ограничением базы.

Пароль проверяем по хэшу

bcrypt использует случайную соль: один пароль может дать разные хэши.

security.py · фрагмент использования — Учебный пример

```
hashed = pwd_context.hash(password)

pwd_context.verify(password, hashed)
# True

pwd_context.verify("wrong", hashed)
# False
```

✓ ПРОВЕРЕНО С PASSLIB + BCRYPT

```
Хэши одного пароля различаются: True
Верный пароль проходит: True
Неверный пароль проходит: False
```

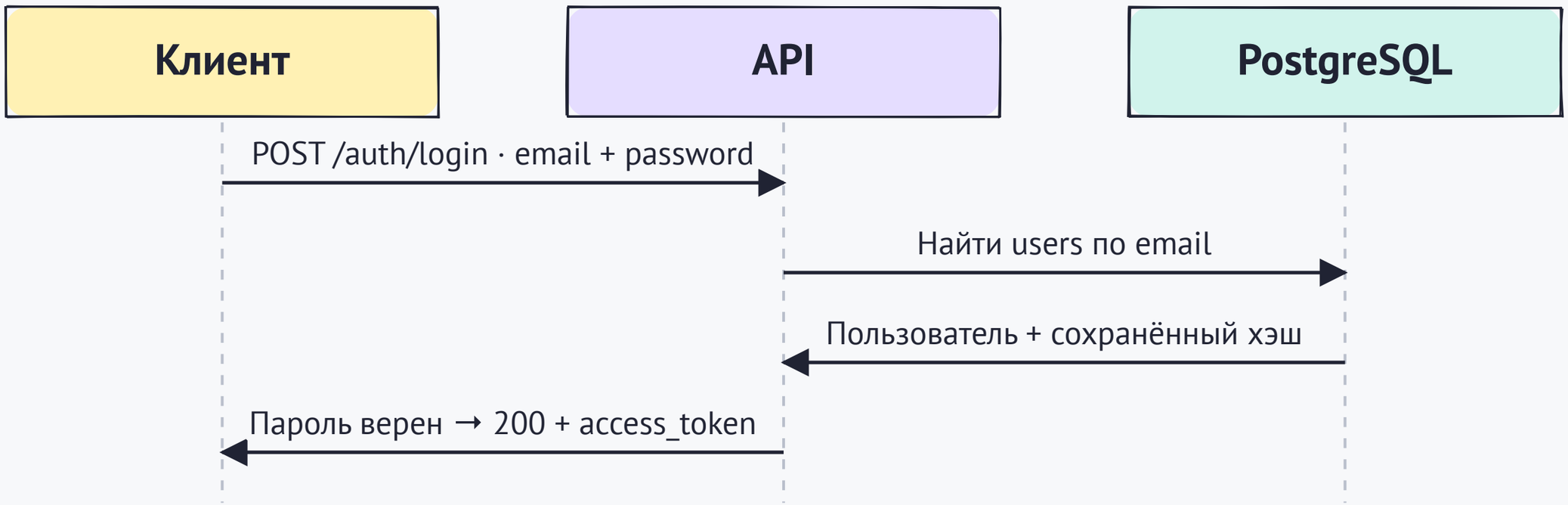
Обратной расшифровки нет

verify повторяет проверку с параметрами из сохранённого хэша.

↳ Храним хэш, а не пароль. Соль не секретна; она сохраняется вместе с параметрами проверки.

Логин выдаёт временный токен

Сначала проверяем пароль, затем создаём подтверждение для следующих запросов.



Неверный пароль или неизвестный email → одинаковый ответ **401**.

↳ Токен выдаём только после проверки. Пароль не кладём в токен и не отправляем в следующих запросах.

JWT: данные и подпись

Подписанный JWT в нашем примере состоит из трёх частей.

`header`
Алгоритм и тип

•

`payload`
Утверждения о доступе

•

`signature`
Проверка целостности

Payload · из проверенного примера — Учебный пример

```
{
  "sub": "7",
  "exp": 1791052678,
  "iss": "sneaker-lesson",
  "aud": "sneaker-api"
}
```

Подпись не скрывает данные

Header и payload можно прочитать.

`exp` — момент истечения в Unix-секундах. Пароля здесь нет.

↳ Проверяем подпись и обязательные поля. Простое декодирование payload не подтверждает подлинность.

Bearer-токен идёт в заголовке

После логина клиент прикладывает токен к защищённому запросу.

Следующий HTTP-запрос · шаблон — Учебный пример

```
GET /auth/me HTTP/1.1
Host: localhost:8007
Authorization: Bearer <access_token>
```

Клиент

Подставляет access_token из ответа логина.

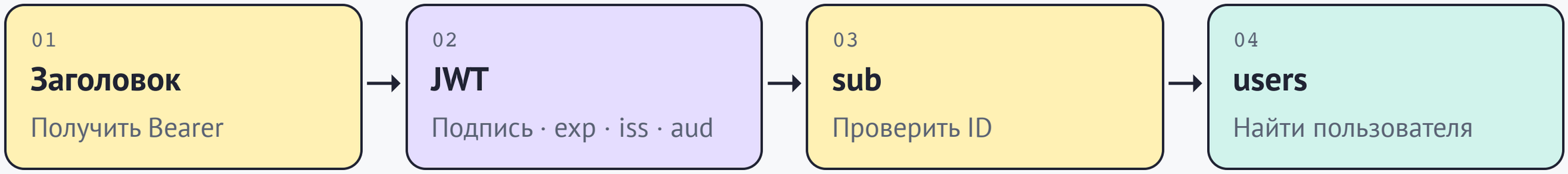
Сервер

Извлекает токен, проверяет его и получает пользователя.

↳ Bearer означает доступ по владению токеном. Его нужно защищать так же внимательно, как данные входа.

get_current_user собирает проверку

Одна зависимость устанавливает пользователя для защищённых маршрутов.



Псевдокод зависимости — Учебный пример

```
claims = проверить_токен(token)
user_id = проверить_ID(claims["sub"])
user = найти_пользователя(user_id)
если user отсутствует: отказ 401
вернуть user
```

Результат зависимости

Объект User из БД.
Следующий шаг проверяет права.

↳ Валидной подписи недостаточно: пользователь должен существовать, а поля токена — пройти проверку.

Администратор — отдельная проверка

Сначала получаем пользователя, затем читаем его роль из базы.

api.py · сокращённый фрагмент — Учебный пример

```
def require_admin(
    user = Depends(get_current_user),
):
    if not user.is_admin:
        raise HTTPException(403)
    return user
```

GET /admin/check	Ответ
Аня · is_admin=false	403
Администратор · is_admin=true	200

Регистрация не выдаёт роль

Публичный запрос с `is_admin=true` отклоняется.

↳ is_admin берём из текущей записи users. Полям запроса и неподтверждённым claims доверять нельзя.

Чужой заказ не становится своим

В запросе должны совпасть и ID заказа, и ID его владельца.

Выборка в GET /orders/{order_id} — Учебный пример

```
stmt = select(Order).where(
    Order.id == order_id,
    Order.user_id == user.id,
)
order = session.scalar(stmt)
if order is None:
    raise HTTPException(404)
```

Аня · user.id = 7

order 501 AND user 7
→ **200 · заказ найден**

Борис · user.id = 8

order 501 AND user 8
→ **404 · строки нет в выборке**

↳ Предсказуемый ID не даёт доступ. Секретность номера заказа не заменяет проверку владельца.

Проверяем не только успешный вход

Меняем одно условие и смотрим, где запрос остановился.

Сценарий	Проверенный ответ
Нет / истёк / неверная подпись	401
Нет пользователя / нет exp / другой алгоритм	401
Действительный токен → /auth/me	200
Обычный пользователь → /admin/check	403
Администратор → /admin/check	200

↳ 401 — не удалось установить пользователя. 403 — пользователь известен, но права не подходят.

Практика: найдите место отказа

Токен → пользователь → права → операция. Где закончится каждый запрос?

Результат	Объяснение
200 · Аня	Токен верен, владелец заказа совпал
404 · Борис	Пользователь найден, заказ ему не принадлежит
401 · истёкший токен	Остановились в get_current_user
403 · обычная роль	Остановились в require_admin

↳ Доступ проверяем до выполнения действия. Следующая глава соединит эти правила с оформлением заказа.

09 Оформление заказа

Корзина ещё не резервирует товар

Выбор покупателя и наличие на складе могут измениться независимо.

01
Аня выбрала
Размер 101 × 2

→

02
Прошло время
Другой покупатель купил пары

→

03
Аня оформляет
Остаток проверяем снова

cart_items.quantity
Сколько пар хочет купить человек.

product_sizes.stock
Сколько пар доступно прямо сейчас.

↳ Добавление в корзину не обещает наличие. Окончательное решение принимается при оформлении.

Сервер собирает данные для покупки

Цена и владелец приходят из доверенных серверных источников.

Что нужно	Откуда берём
Пользователь	Проверенный токен → users
Выбранные размеры и количество	Его строки cart_items
Текущий остаток	product_sizes.stock
Цена одной пары	products.price

Тело запроса · опасная подмена — Учебный пример

```
{"user_id": 8, "total": "1.00"}
# Эти значения не назначают
# владельца и сумму заказа
```

Операция оформления

Берём текущего пользователя.
Сумму считаем сами.

↳ Клиент просит оформить свою корзину. Сервер определяет владельца, состав и итог.

Перед записью проверяем условия

Оформление допустимо только для целой корректной корзины.

1. Пользователь установлен	Работаем только с его корзиной
2. Корзина не пустая	Есть хотя бы одна позиция
3. Количество допустимо	Для каждой строки quantity > 0
4. Размер существует	Ссылка ведёт к нужному товару
5. Остатка хватает	Для каждой позиции stock ⩾ quantity

↳ Если не проходит хотя бы одна позиция, весь заказ отклоняется без частичного списания.

09 Оформление заказа

Сумму считаем по каждой позиции

Количество умножаем на цену одной пары, затем складываем результаты.

Размер ID	Цена из products	Количество	Стоимость позиции
101	7 990.00	2	15 980.00
201	5 990.00	1	5 990.00

15 980.00 + 5 990.00 **21 970.00 ₽**

`total = Σ(price × quantity)` · точные Decimal / numeric

↳ Все цены — из базы. В этом примере нет доставки, скидок и налогов.

09 Оформление заказа

Заказ запоминает цену покупки

История не пересчитывается при изменении каталога.

Во время покупки

product 10 → **7 990 ₽**
Сохраняем в price_at_purchase.

Позже в каталоге

products.price → **8 990 ₽**

Сохранённая позиция заказа — Учебный пример

```
product_size_id: 101
quantity: 2
price_at_purchase: 7990.00
```

История покупки

Стоимость позиции: **15 980 ₽**
Итог заказа: **21 970 ₽**

↳ price_at_purchase — снимок цены одной пары. total фиксирует согласованный итог заказа.

09 Оформление заказа

Транзакция объединяет изменения

Заказ, его состав, остатки и корзина должны измениться согласованно.

ОДНА ТРАНЗАКЦИЯ

`orders`	`order_items`	`product_sizes`	`cart_items`
Создать заказ	Записать состав и цены	Списать пары	Очистить свою корзину

COMMIT

Все изменения зафиксированы.

ROLLBACK

Изменения этой операции отменены.

↳ Не допускаем «заказ есть, а склад не списан» или «корзина пуста, но заказ не создан».

Оформление по шагам

Текущий пользователь уже установлен. Одна операция владеет транзакцией.

Псевдокод · последовательность — Учебный пример

```
начать транзакцию
заблокировать владельца корзины
прочитать его корзину
заблокировать размеры; прочитать цены
проверить все позиции; посчитать итог
создать заказ и позиции
списать остатки; очистить корзину
зафиксировать; при ошибке откатить
```

Граница одна

Внутренние функции не делают собственный commit.

После ожидания читаем актуальное

Проверяем остаток уже после получения блокировки.

↳ Сначала согласованное чтение и проверки, затем записи. Успех возвращаем после commit.

09 Оформление заказа

После commit виден целый заказ

Сравним состояние до и после оформления корзины Ани.

Что проверяем	До	После commit
Новый заказ	Нет	№ 502 · created
Новые позиции	0	2 · цены 7990.00 и 5990.00
Остатки 101 / 201	3 / 5	1 / 4
Корзина Ани	2 позиции	Пустая
Итог нового заказа	—	21970.00

✓ Проверено в PostgreSQL. Корзина Бориса и прежний заказ 501 сохранены.

↳ Проверяем не только результат функции: читаем заказ, состав, остатки и корзину из базы.

09 Оформление заказа

Ошибка посередине: полный откат

Сбой специально вызван после уже выполненных изменений.

Место ошибки	Новых заказов	Корзина Ани
После INSERT orders	0	2 строки
После списания первой позиции	0	2 строки
После DELETE корзины	0	2 строки

Во всех трёх опытах остатки восстановлены: 101 → 3, 201 → 5.

↳ ROLLBACK отменяет изменения текущей операции даже после INSERT, UPDATE и DELETE.

09 Оформление заказа

Двое увидели последнюю пару

Две отдельные транзакции читают stock=1 до чужого commit.

Шаг	Аня · соединение A	Борис · соединение B
1	Читает stock = 1	Читает stock = 1
2	В памяти: 1 − 1 = 0	В памяти: 1 − 1 = 0
3	Пишет stock = 0, создаёт заказ	Ждёт UPDATE, затем тоже пишет 0
4	COMMIT · покупка прошла	COMMIT · покупка тоже прошла

Была 1 пара → продали 2 В базе stock = 0. Даже CHECK(stock ⩾ 0) не заметил проблему.

↳ Каждая транзакция может быть атомарной и всё равно принимать решение по устаревшему чтению.

Блокируем строку и проверяем снова

Один покупатель завершает работу с остатком, второй ждёт.

Шаг	Аня · первая получила строку	Борис · ждёт ту же строку
1	SELECT … FOR UPDATE → stock=1	SELECT … FOR UPDATE → ожидание
2	Проверяет 1 ⩾ 1; списывает до 0	Ещё не принимает решение о покупке
3	COMMIT · блокировка снята	Получает актуальный stock=0
4	Один новый заказ сохранён	Отказ: остатка не хватает

Блокирующее чтение внутри транзакции — Учебный пример

```
SELECT id, stock FROM product_sizes
WHERE id = 101 FOR UPDATE;
```

↳ В опыте READ COMMITTED: второй покупатель после ожидания видит 0 и не создаёт заказ.

Проверяем границы защиты

Последняя пара — только один из обязательных сценариев.

Сценарий	Проверенный результат
Пустая / не хватает второй позиции	Отказ, изменения не сохраняются
Два оформления одного пользователя	Один заказ; второе видит пустую корзину
Изменение корзины ждёт оформление	Новая позиция остаётся после commit

> Потерянный ответ после commit — отдельный случай: повтор запроса требует политики идемпотентности.

↳ Все операции одной корзины должны соблюдать общий протокол. Блокировки склада для этого недостаточно.

Практика: собираем магазин по модулям

После каждого модуля должен появиться результат, который можно проверить.

M0
Окружение
/docs и запрос к БД

M1
Схема и данные
Миграции + seed

M2
Каталог
Чтение, фильтры, CRUD

M3
Вход и права
JWT + защита изменений

M4
Корзина
Данные текущего пользователя

M5
Заказы
Сумма, остатки, транзакция

M6
Завершение
Проверки + README + сдача

↳ Движемся M0 → M6. Один пройденный модуль становится опорой для следующего.

Один модуль делим на малые задачи

Пример M0: четыре шага, у каждого свой проверяемый выход.

1. Python и venv — Файлы зависимостей

Нужный интерпретатор; пакеты установлены

2. PostgreSQL в Compose — `compose.yaml, .env.example`

Контейнер готов; SQL выполняется

3. Каркас FastAPI — `app/main.py,` настройки

Открывается /docs

4. Подключение и /health — `database.py,` обработчик

200 при доступной БД; 503 при сбое

↳ Следующую задачу начинаем после проверки текущей. Размер шага — один понятный результат.

10 Практика с нейросетью

Промпт описывает задачу и проверку

Помощнику нужны конкретные условия текущего проекта.

Цель

Какое поведение должно появиться

Контекст

Что уже есть и какие файлы приложены

Ограничения

Стек, права, границы изменения

Результат

Какие файлы и diff ожидаем

Приёмка

Команды, ответы и состояние данных

Объяснение

Почему решение работает

↳ «Добавь настоящий /health и проверь отказ БД» задаёт более проверяемую цель, чем «сделай всё хорошо».

10 Практика с нейросетью

Готовый промпт для одного шага

Задача M0: /health подтверждает настоящее подключение к базе.

Короткий вариант · полный текст в HTML — Учебный пример

```
Добавь GET /health в текущий FastAPI.
Используй существующий Engine.
Выполни SELECT 1 с таймаутами.
БД доступна → 200, status: ok.
Ошибка доступности → 503.
Покажи diff и команды проверки.
```

Что приложить

main.py · database.py · настройки
requirements.txt · compose.yaml

Что принять

Рабочая БД → успех.
Учебная БД недоступна → отказ.
После восстановления → успех.

↳ Это условия будущей реализации M0. Возврат постоянного {status: ok} их не выполняет.

Передаём нужный контекст

Помощник должен видеть текущие файлы и воспроизводимый сценарий.

Код и договорённости

Дерево проекта · относящиеся файлы
Версии · контракт API · ограничения

Как воспроизвести

Рабочая папка · команда запуска
Запрос · ответ · состояние БД

Пример обезличенного контекста — Учебный пример

```
API: Python на компьютере
БД: сервис db в Docker
Хост: 127.0.0.1:55432
DATABASE_URL: <REDACTED>
JWT_SECRET: <REDACTED>
Токен: <REDACTED>
```

↳ Показываем структуру и существенные настройки. Реальные пароли, ключи и токены заменяем.

Читаем diff до запуска

Проверяем, что новая строка выполняет нужное действие.

Упрощённый фрагмент diff · идея — Учебный пример

```
 def health():
+    with engine.connect() as conn:
+        conn.execute(text("SELECT 1"))
     return {"status": "ok"}
```

Что изменилось

Появился настоящий SQL-запрос.
Соединение закрывается через with.

Чего фрагмент ещё не доказывает

Откуда engine и text?
Где таймауты и ответ 503?

↳ Читаем не только добавленные строки: проверяем импорты, вызовы, удалённое поведение и ошибки.

Проверка заканчивается наблюдением

Команда запуска, ответ и данные должны подтверждать одно поведение.

01
Папка
Где выполняю команду

→

02
Запуск
Что именно запускаю

→

03
Ответ
Статус + тело

→

04
Данные
Что изменилось в БД

Воспроизводимый опыт из этой главы — Учебный пример

```
cd presentation/sneaker-checkout-examples
python verify.py
```

Фактический результат опыта

23 проверки пройдены.
Откаты, гонки, корзина и остатки проверены в PostgreSQL.

↳ Условия установки и запуска — в инструкции примеров. Один код ответа не доказывает все изменения.

10 Практика с нейросетью

Сообщаем об ошибке воспроизводимо

Нужно объяснить разницу между ожиданием и фактом.

Карточка проблемы · пример — Учебный пример

```
Шаг: M0, проверка /health
API на компьютере; БД в Docker
Ожидал: 200, {status: ok}
Получил: 503, connection refused
Адрес: 127.0.0.1:5432
Публикация БД: 55432 → 5432
```

Проверяем гипотезу

Порт в адресе API отличается от опубликованного порта компьютера.

После исправления

Повторить исходный запрос.
Проверить отказ и восстановление БД.

↳ Описываем сценарий, ошибку и относящийся код. Исправляем причину, затем повторяем проверку.

10 Практика с нейросетью

Отдельное ревью ищет пропущенное

Успешный запрос Ани ещё не доказывает защиту её заказа.

Условие выборки после исправления — Учебный пример

```
select(Order).where(
    Order.id == order_id,
    Order.user_id == user.id,
)
```

Повторная проверка

Аня → заказ 501: **200**
Борис → заказ 501: **404**
Без действительного токена: **401**

↳ Ревью сверяет код с требованиями и контрпримерами. После исправления повторяем проверку двумя аккаунтами.

Переходим дальше после приёмки

Нужно уметь запустить результат, проверить его и объяснить.

Работает

Положительный сценарий воспроизведён

Отказывает правильно

Граничный случай не ломает данные и права

Понятно

Могу объяснить путь запроса и новую часть кода

Сохранено

Diff просмотрен; код и результаты зафиксированы

Короткая запись о шаге	Что сохранить
Цель и файлы	Что изменили и зачем
Команды и факт	Что запустили; что получили
Осталось	Непроверенное и следующий маленький шаг

↳ Следующая глава — практика M0: начинаем с окружения и доводим подключение к БД до проверяемого результата.

M0: запускаем API и проверяем базу

Начало практики: отдельная папка sneaker_shop и один проверяемый маршрут.

Наблюдение	Что доказывает
/docs открывается	Приложение отвечает
GET /health → 200	SQL-запрос к БД выполнен
БД остановлена → 503	Проверка не выдаёт ложный успех

Где что работает

Python + Uvicorn — на компьютере.
PostgreSQL — в Docker.

Результат главы

Файлы запуска, venv, Compose, конфигурация и GET /health.

↳ M0 принят, когда вы можете запустить всё по инструкции и показать отказ и восстановление БД.

11 Практика M0 · окружение

Шаг 1. Папка, Python и свой venv

Выберите команды для своей оболочки. Не смешивайте системный Python и окружение проекта.

macOS / Linux — Учебный пример

```
mkdir sneaker_shop
cd sneaker_shop
python3 --version
python3 -m venv .venv
source .venv/bin/activate
```

Windows · PowerShell — Учебный пример

```
mkdir sneaker_shop
cd sneaker_shop
py -3 --version
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

↳ Проверка: python -c "import sys; print(sys.executable)" показывает Python внутри .venv.

Шаг 2. Фиксируем файлы запуска

Конфигурация, зависимости и локальные секреты имеют разные роли.

Файл	Зачем он нужен
requirements.txt	Точные версии пакетов → повторяемая установка
.env.example	Шаблон переменных с вымышленными значениями
.env	Локальные значения; не добавляем в Git
.gitignore	.env, .venv/, __pycache__/ не попадают в коммит
app/core/config.py	Python загружает .env из корня проекта

После подготовки requirements.txt — Учебный пример

```
python -m pip install -r requirements.txt
python -m pip check
```

↳ В M0–M1 устанавливаем пакеты для запуска, БД и миграций. Зависимости авторизации добавим в M3.

Шаг 3. Просим настроить PostgreSQL

Промпт задаёт сервис, адрес, хранение данных и критерий готовности.

compose.yaml · фрагмент сервиса db — Учебный пример

```
db:
  image: postgres:17
ports:
  - "127.0.0.1:55433:5432"
volumes:
  - pgdata:/var/lib/postgresql/data
# environment и healthcheck
# см. полный compose.yaml
```

Что поручаем нейросети

Compose + .env.example + .gitignore.
База sneaker_shop; проверка pg_isready.

Что читаем в diff

Порт компьютера 55433.
Именованный том pgdata.
Пароль берётся из окружения.

↳ API с компьютера подключается к 127.0.0.1:55433. Адрес db:5432 относится к сети Compose.

Шаг 4. Поднимаем и проверяем БД

В корне проекта уже есть compose.yaml и заполненный .env.

Терминал · из корня проекта — Учебный пример

```
docker compose config --quiet
docker compose up -d --wait
docker compose ps
docker compose exec db psql \
  -U practice -d sneaker_shop \
  -c "SELECT 1;"
```

Что наблюдаем	Ожидаемый результат
Сервис db	running / healthy
Публикация порта	127.0.0.1:55433 → 5432
SELECT 1	Одна строка со значением 1

Следующая проверка

psql работает внутри контейнера.
Подключение Python проверим отдельно.

↳ Успешный запрос внутри контейнера подтверждает БД. Путь от FastAPI ещё предстоит проверить.

Шаг 5. API выполняет настоящий запрос

В полном примере Engine создаётся один раз; /health использует его соединение.

app/main.py · обработчик — Учебный пример

```
@app.get("/health")
def health():
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
    except (OperationalError, TimeoutError):
        return JSONResponse(status_code=503,
            content={"status":
"unavailable"})
    return {"status": "ok"}
```

Запуск из корня проекта — Учебный пример

```
python -m uvicorn app.main:app \
  --reload --port 8000
```

Открыть в браузере

127.0.0.1:8000/docs
GET /health → Execute

↳ config.py читает адрес → database.py создаёт Engine → main.py выполняет SELECT 1.

Если M0 не запускается: проверяем слой

Идём от конкретного сообщения к одной проверке, затем повторяем запрос.

Симптом	Вероятная причина	Следующий шаг
ModuleNotFoundError	Другой Python / нет пакета	sys.executable; python -m pip show fastapi
address already in use	Порт занят другим процессом	Выбрать свободный порт и обновить URL
connection refused	БД не готова / неверный порт	compose ps; сверить 55433 в DATABASE_URL
password authentication failed	Реквизиты не совпадают	Сверить пользователя и пароль существующей БД
Таблиц ещё нет	M0 не создаёт схему	В M1 применить миграцию; не добавлять create_all

↳ Зафиксируйте папку, команду и ошибку. Исправьте одну причину и снова проверьте /health.

Принимаем M0: успех → отказ → успех

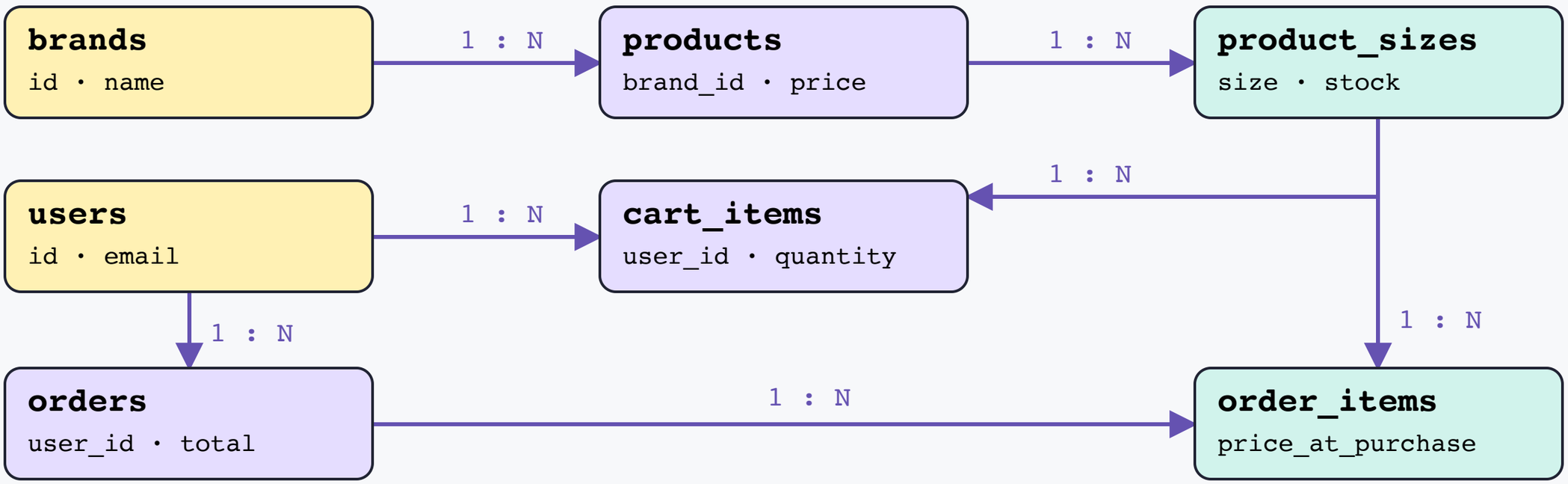
Эти ответы получены от настоящего Uvicorn и PostgreSQL, без подмены БД.

Действие	GET /health	JSON
БД работает	200	{"status":"ok"}
docker compose stop db	503	{"status":"unavailable"}
docker compose up -d --wait	200	{"status":"ok"}

↳ Остановку и восстановление выполняем в другом терминале; процесс API продолжает работать.

M1: сначала рисуем семь таблиц

Восстанавливаем связи до генерации кода. Стрелка: одна строка → много связанных строк.



У cart_items и order_items есть product_size_id. Остаток хранится в product_sizes; снимок цены — в order_items.

↳ Для каждого поля решаем: тип, NULL, значение по умолчанию, ключ и ограничение.

Шаг 1. Модели каталога

Попросите три класса: Brand, Product, ProductSize — на одном Base.

Product · фрагмент модели — Учебный пример

```
brand_id: Mapped[int] = mapped_column(
    ForeignKey("brands.id"))
price: Mapped[Decimal] = mapped_column(
    Numeric(10, 2))
brand: Mapped["Brand"] = relationship(
    back_populates="products")
```

Ограничения таблиц

price > 0 · stock ⩾ 0
UNIQUE(product_id, size)

Два разных механизма

ForeignKey — ссылка в БД.
relationship — переход в Python.

↳ Проверяем импорт трёх моделей и metadata. Таблицы появятся после Alembic upgrade.

Шаг 2. Пользователь и его корзина

Связи и ограничения описываем сейчас; вход и права API появятся в M3–M4.

CartItem · схема связей и Python — Учебный пример

```
user_id → ForeignKey("users.id")
product_size_id →
    ForeignKey("product_sizes.id")

UniqueConstraint("user_id",
    "product_size_id", name="uq_cart_size")
CheckConstraint("quantity > 0",
    name="ck_cart_quantity")
```

User

email — уникальный.
hashed_password — обязательный.
is_admin — false по умолчанию.

CartItem

Одна строка на пользователя и размер.
quantity не может быть 0.

↳ user_id связывает данные, но сам по себе не запрещает HTTP-запрос к чужой корзине.

Шаг 3. Заказ и позиции заказа

Разделяем общий документ и строки покупки. Цена одной пары фиксируется отдельно.

Order	OrderItem
user_id → users.id	order_id → orders.id
status = created	product_size_id → product_sizes.id
total: Numeric(12,2)	price_at_purchase: Numeric(10,2)
created_at с часовым поясом	quantity > 0

Связь в Python

order.items → список позиций.
item.order → один заказ.

Снимок цены

2 пары × 7 990 = 15 980.
price_at_purchase = 7 990.

↳ Модель описывает хранение. Расчёт total и транзакцию оформления реализуем в M5.

Шаг 4. Модели превращаем в миграцию

Пустая учебная БД + импорт всех моделей → кандидат ревизии → чтение → применение.

1 **Подключить metadata**

```
from app.models import Base
target_metadata = Base.metadata
```

2 **Создать кандидата**

```
python -m alembic revision --
autogenerate -m initial
```

3 **Прочитать файл**

```
7 create_table • поля • FK
UNIQUE • CHECK • nullable
```

4 **Применить и сверить**

```
python -m alembic upgrade head
python -m alembic current
```

↳ revision создаёт файл. upgrade меняет БД. В готовом эталоне ревизия 0001 уже есть.

Шаг 5. Seed заполняет каталог

Фиксированные данные помогают сравнить результат. Повторный запуск не сбрасывает остатки.

После миграции · два запуска — Учебный пример

```
python -m app.seed
python -m app.seed
```

Таблица	После 1-го	После 2-го
brands	2	2
products	3	3
product_sizes	4	4

Порядок вставки

Бренды → товары → размеры.
North Runner: 42 → 3, 43 → 0.

Чего seed не меняет

Существующие цены и остатки.
Пользователи и заказы пока пусты.

↳ 25 проверок M1 включают повтор seed, сохранение изменённого остатка и корректную генерацию новых ID.

Шаг 6. Проверяем с нуля и на ошибках

Новая отдельная база доказывает воспроизводимость миграции и seed.

Автопроверка готового эталона — Учебный пример

```
python verify.py
```

Что делает сценарий

Создаёт временную БД.
upgrade → seed × 2 → проверки.
Удаляет только свою временную БД.

Проверяем	Фактический результат
Схема	7 таблиц + alembic_version
Каталог	2 бренда / 3 товара / 4 размера
ORM-переход	North Runner → North → 42, 43
Ошибочные записи	12 отказов ограничений
Повтор и ID	Нет дубликатов; новый ID свободен

↳ Проверка на вашей старой БД не заменяет запуск с нуля. Рабочую базу и том для этого не удаляем.

Принимаем M1: объясни три результата

Схема создаётся миграцией, данные — seed, код обращается к ним через ORM.

Модель

Описывает таблицу в Python. Правка класса сама не меняет БД.

Миграция

Применяет изменения схемы. Создаёт таблицы и ограничения.

Seed

Добавляет данные каталога. Повтор сохраняет строки и stock.

Приёмка: upgrade с нуля ✓ · 7 таблиц ✓ · каталог 2/3/4 ✓ · повтор seed ✓ · отрицательные проверки ✓

↳ Перед M2 сохраните код, команды запуска и фактические результаты. Умейте объяснить каждый шаг.